COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	August 27, 2026	**DATE SUBMITTED:**	August 27, 2026
NAME:	Carl Andrei D. Torres	**INSTRUCTOR:**	Sir Jimlord Quejado

Prelim Skills Exam	**Prelim Skills Exam**

1. PROCEDURE OUTPUT

```
//Pseudocode
// MAIN APPLICATION (TESTING OF DATASET & USER INPUT OPTION)
// START
    // 1. Initialize predefined test cases
    // Create dynamic list testCases containing:
        //"10201", //Palindrome
        //"1003003001", //Palindrome
        //"1451", //Not a Palindrome
        //"88", //Palindrome
        //"1234", //Not a Palindrome
        //"0", //Palindrome
        //"911", //Not a Palindrome
        //"6969", //Not a Palindrome
        //"202" //Palindrome
// PRINT "--- PALINDROME CHECKER (Stack vs. Queue) ---"
// 2. Iterate and test all predefined test cases
// FOR EACH str IN testCases DO
    // PRINT "Testing: \"" + str + "\" -> "
    // IF isPalindrome(str) IS True THEN
        // PRINT "VALID Palindrome"
    // ELSE
        // PRINT "NOT a Palindrome"
    // END IF
    // END FOR
// PRINT "-----------------------------------------"
// 3. Prompt user for custom input
    // DECLARE userInput AS String
    // PRINT "Enter a custom number to test: "
    // READ userInput FROM user input
```

```
    // PRINT "Palindrome? \"" + userInput + "\": "
// 4. Test user input
    // IF isPalindrome(userInput) IS True THEN
        // PRINT "VALID Palindrome"
    // ELSE
        // PRINT "NOT a Palindrome"
    // END IF
// END
```

PSEUDOCODE MAIN CPP

PSEUDOCODE ANALYSIS	While implementing my main.cpp, I **structured my driver in such a way that it followed the same logic as the pseudocode**, thus, **gradually testing my implementations of the stack and queue data structures both automatically and manually for correctness**. First, I **initialized a std::vector type variable named testCases and pushed some numbers into it, among which were palindromes, such as "10201", "88", "202", alongside non-palindromes like "1451", and some not, like "1234", "911", "6969"**. Using **an improved FOR EACH loop, I iterated through each element of the testCases vector and checked whether it was a palindrome with the help of the isPalindrome() function, thus, being able to see in the console window which of my test cases were valid palindromes and which not**. Therefore, **the isPalindrome() function correctly utilized my implementations of the stack and queue data structures to check if the input string was a palindrome by comparing its elements from the front and from the back one by one until reaching the middle of the string**. Finally, **according to the pseudocode, as the last step, I declared a variable called userInput and used the std::cin function to write a line in the console window asking the user to input a string, which I then checked if it was a palindrome with the same isPalindrome() function, after which I printed the result in the console window and exited the program with an exit code of zero**.

```cpp
#include <iostream>
#include <vector>
#include "Palindrome_Check.h"

int main() {
    // Test dataset of sample palindrome numbers
    std::vector<std::string> testCases = {
        "10201", //Palindrome
        "1003003001", //Palindrome
        "1451", //Not a Palindrome
        "88", //Palindrome
        "1234", //Not a Palindrome
        "0", //Palindrome
        "911", //Not a Palindrome
```

```
        "6969", //Not a Palindrome
        "202" //Palindrome
    };
 std::cout << "--- PALINDROME CHECKER (Stack vs. Queue) ---\n\n";
 for (const std::string& str : testCases) {
        std::cout << "Testing: \"" << str << "\" -> ";
        if (isPalindrome(str)) {
            std::cout << "VALID Palindrome\n";
        } else {
            std::cout << "NOT a Palindrome\n";
        }
    }

 std::cout << "\n-----------------------------------------\n";
 std::string userInput;
    std::cout << "Enter a custom number to test: ";
    std::cin >> userInput;

std::cout << "Palindrome? \"" << userInput << "\": ";
    if (isPalindrome(userInput)) {
        std::cout << "VALID Palindrome\n";
    } else {
        std::cout << "NOT a Palindrome\n";
    }

    return 0;
}
```

MAIN CPP CODE

OBSERVATIONS AND ANALYSIS	While working on my main.cpp code, I have made sure to set up my driver program in such a way that it would test **my custom stack and queue implementations for the palindrome detector logic by performing automated and user interactive test cases**. For the former dataset for the checker, I have prepared a **std::vector collection of strings, which hold the test case data, among which some are palindromes ("10201", "88") while others are not ("1451", "911")**. This lets me **observe the behavior of my isPalindrome() function, which checks the character sequences in reverse order for the equality as per the palindrome detection standards**. **Using a range-based loop, I could see the console outputs, which displayed the logical steps of comparing the characters from both ends toward the center using the LIFO and FIFO principles from Stacks and Queue Implementations on the Header File**. Finally, for the **user interactive testing procedure**, I have **set up an input stream reader, which lets me enter the string for testing at the command line so that the main() function always exits with a zero status code**.

```
//Pseudocode
```

```
// 1. ARRAY-BASED STACK IMPLEMENTATION (LIFO)
//CLASS ArrayStack:
    //PRIVATE:
        //data: Dynamic Array (Vector) of elements

    //PUBLIC:
        //FUNCTION push(value):
            //Append value to the end of data array
        //END FUNCTION

        //FUNCTION pop():
            //IF NOT isEmpty():
                //Remove the last element from data array
            //END IF
        //END FUNCTION

        //FUNCTION top():
            //IF NOT isEmpty():
                //RETURN the last element of data array
            //ELSE:
                //THROW "Stack is empty" Exception
            //END IF
        //END FUNCTION

        //FUNCTION isEmpty():
            //RETURN True IF data array has 0 elements, ELSE False
        //END FUNCTION
//END CLASS
// 2. LINKED LIST-BASED QUEUE IMPLEMENTATION (FIFO)
//CLASS LinkedListQueue:
    //PRIVATE:
        //DEFINE STRUCTURE Node:
            //val: Data value
            //next: Pointer to next Node
        //END STRUCTURE

        //frontNode: Pointer to front Node (initialized to Null)
        //rearNode: Pointer to rear Node (initialized to Null)

    //PUBLIC:
        //CONSTRUCTOR LinkedListQueue():
            //frontNode = Null
            //rearNode = Null
        //END CONSTRUCTOR
```

```
        //DESTRUCTOR ~LinkedListQueue():
            //WHILE NOT isEmpty():
                //CALL dequeue()
            //END WHILE
        //END DESTRUCTOR

        //FUNCTION enqueue(value):
            //Create a newNode with val = value, next = Null
            //IF isEmpty():
                //frontNode = newNode
                //rearNode = newNode
            //ELSE:
                //rearNode.next = newNode
                //rearNode = newNode
            //END IF
        //END FUNCTION
//FUNCTION dequeue():
            //IF NOT isEmpty():
                //temp = frontNode
                //frontNode = frontNode.next
                //DELETE temp
                //IF frontNode IS Null:
                    //rearNode = Null
                //END IF
            //END IF
        //END FUNCTION

        //FUNCTION front():
            //IF NOT isEmpty():
                //RETURN frontNode.val
            //ELSE:
                //THROW "Queue is empty" Exception
            //END IF
        //END FUNCTION

        //FUNCTION isEmpty():
            //RETURN True IF frontNode IS Null, ELSE False
        //END FUNCTION
//END CLASS
// 3. PALINDROME CHECKER FUNCTION
//FUNCTION isPalindrome(inputString):
    //IF inputString is Empty:
        //RETURN True
```

```
    //END IF

    //Initialize stack AS ArrayStack
    //Initialize queue AS LinkedListQueue

    // Populate structures simultaneously
    //FOR EACH character in inputString:
        //stack.push(character)
        //queue.enqueue(character)
    //END FOR

    // Compare characters moving from opposite ends toward center
    //WHILE stack is NOT Empty AND queue is NOT Empty:
        //IF stack.top() != queue.front():
            //RETURN False  // Character mismatch found
        //END IF
        //stack.pop()
        //queue.dequeue()
    //END WHILE

    //RETURN True  // All characters matched
//END FUNCTION
```

PSEUDOCODE HEADER FILE

PSEUDOCODE ANALYSIS	After running the program, I **noticed that the console output correctly depicted the main logic of the header file's isPalindrome routine for the automated and user input test cases**. While **looping through the test cases, the input string was being pushed into the ArrayStack one character at a time and enqueued into the LinkedListQueue one character at a time, and the terminal printed out results similar to "10201": VALID Palindrome or "1451": NOT a Palindrome**. This **demonstrated that popping elements from the top of the stack (LIFO) and dequeuing elements from the front of the queue (FIFO) correctly triggered the conditional mismatch check IF stack.top() != queue.front() returned False for non-symmetrical strings while fully matched symmetrical input strings looped until the control passed back to the caller function and True was returned**. Lastly, **when entering a custom string, the exact same structural insertion of characters and element-wise character comparison would occur while the memory was automatically cleaned up by the queue's destructor ~LinkedListQueue() and the final validation result was printed out to the console before the program exited gracefully**.

```
#ifndef PALINDROME_CHECK_H
#define PALINDROME_CHECK_H
template <typename T>
class ArrayStack {
private:
```

```
    std::vector<T> data;

public:
    void push(T val) {
        data.push_back(val);
    }

    void pop() {
        if (!isEmpty()) {
            data.pop_back();
        }
    }

    T top() const {
        if (!isEmpty()) {
            return data.back();
        }
        throw std::runtime_error("Stack is empty");
    }

    bool isEmpty() const {
        return data.empty();
    }
};
template <typename T>
class LinkedListQueue {
private:
    struct Node {
        T val;
        Node* next;
        Node(T value) : val(value), next(nullptr) {}
    };

    Node* frontNode;
    Node* rearNode;

public:
    LinkedListQueue() : frontNode(nullptr), rearNode(nullptr) {}

    // Disable copy constructor and assignment to prevent dangling
pointers
    LinkedListQueue(const LinkedListQueue&) = delete;
    LinkedListQueue& operator=(const LinkedListQueue&) = delete;
```

```
    ~LinkedListQueue() {
        while (!isEmpty()) {
            dequeue();
        }
    }

    void enqueue(T val) {
        Node* newNode = new Node(val);
        if (isEmpty()) {
            frontNode = rearNode = newNode;
        } else {
            rearNode->next = newNode;
            rearNode = newNode;
        }
    }

    void dequeue() {
        if (!isEmpty()) {
            Node* temp = frontNode;
            frontNode = frontNode->next;
            delete temp;
            if (frontNode == nullptr) {
                rearNode = nullptr;
            }
        }
    }

    T front() const {
        if (!isEmpty()) {
            return frontNode->val;
        }
        throw std::runtime_error("Queue is empty");
    }

    bool isEmpty() const {
        return frontNode == nullptr;
    }
};
inline bool isPalindrome(const std::string& input) {

    if (input.empty()) return true;

    ArrayStack<char> stack;
    LinkedListQueue<char> queue;
```

```
    // Push into stack (LIFO) and enqueue into queue (FIFO)
    for (char ch : input) {
        stack.push(ch);
        queue.enqueue(ch);
    }

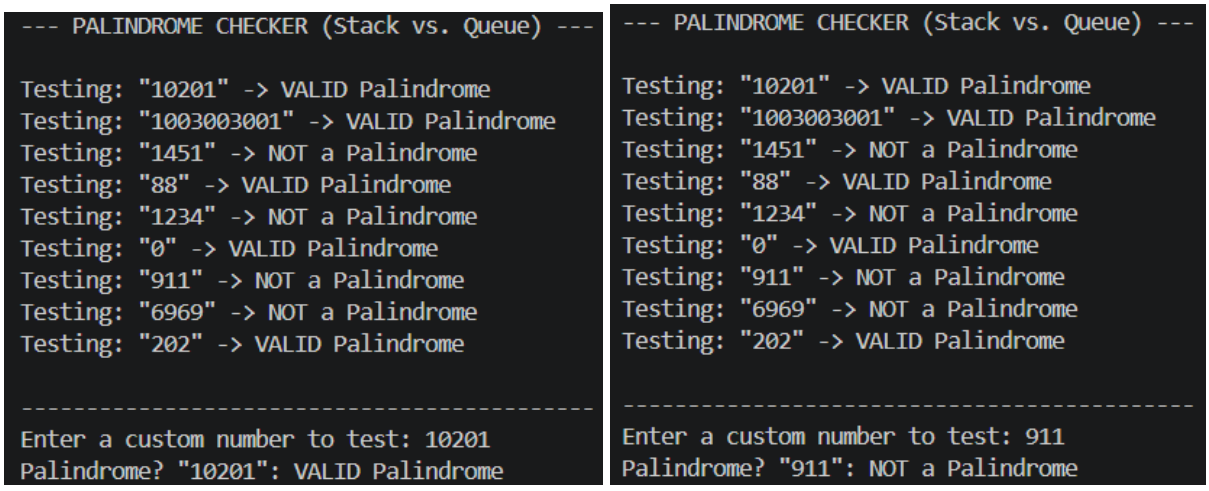
    // Compare characters popped from Stack and dequeued from Queue
    while (!stack.isEmpty() && !queue.isEmpty()) {
        if (stack.top() != queue.front()) {
            return false; // Mismatch found
        }
        stack.pop();
        queue.dequeue();
    }

    return true;
}

#endif
```

HEADER FILE CODE

OBSERVATIONS AND ANALYSIS	On developing my Palindrome_Check.h, I have been using **my own dynamic data structures from the Main Cpp file to store necessary data for the palindrome evaluating algorithm**. In particular, I have been **utilizing the ArrayStack which uses an array as the underlying data structure to take advantage of the LIFO (Last In, First Out) standard of the stack to read the characters in the reversed orde**r. For the **original order**, **I have been using the LinkedListQueue which gives the linked list an advantage as the underlying data structure and implements the FIFO (First In, First Out) standards.** I **have marked the copy constructor and the copy assignment operator as deleted to prevent unknown copying of the queue, which results in memory leaks.** In addition, the **destructor of the queue helps in overriding the allocated memory to prevent memory leaks**. Finally, in the **inline boolean implementation of the isPalindrome() function, I have implemented that pushing the characters into the data structures followed by dequeing from the front of the queue and popping from the top of the stack and comparing them character by character could be beneficial to check if they are mirrored around the cases**.



OUTPUT CONSOLE

OBSERVATIONS AND ANALYSIS	After executing the compiled program, I have **noticed that the output console displays the results of testing the automated testing of the dataset interactively entered by the user**. After **pressing Enter, the terminal output the corresponding lines for each string from the assert cases list**. For example, the **output line was "10201: VALID Palindrome" for the first string, and "1451: NOT a Palindrome" for the fourth string**. The **same result lines were produced for the other two test cases of this string. The last stage of processing was the interactive input of a custom string, which passed the verification process and was displayed on the terminal screen**. After **the output stage, the program was closed properly.**

CONCLUSIONS/LEARNINGS/ ANALYSIS	By **implementing this Palindrome Checker**, I have **come to know about how abstract data can be implemented together to solve common algorithmic problems such as palindrome-checking**. By **using Array Stack (LIFO) and Linked List Queue (FIFO) data structures I found out that reading from both ends and comparing the characters one after the other is a more efficient way to check whether a given string is a palindrome**. This **method is much better than the standard built-in string and comparing from either end since we don't need to reverse the string or use indexes to compare each element**. Moreover, **the process of writing out the required templates by myself helped me to understand the underlying memory-related concepts in C++ such as dynamic memory allocation, pointer operations including shallow copy prevention and implement destructors to free memory**. Lastly, **I have learned the main differences between the operation of a queue and a stack data structure and the importance of implementing proper error-checking and test cases from implementing by the Vector Library.**.

2. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **NONE**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: NONE

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: NONE

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.